



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 06 - Part 2

Linked Lists





THIS WEEK'S AGENDA

- 01. Remove from Front
- 02. Remove from Rear
- 03. Deletion in General
- 04. The Find Function
- 05. Other Linked Data Structures





REMOVE FROM FRONT





REMOVE FROM FRONT

```
void removeFromFront() {  
    if (head == nullptr) {  
        std::cout << "List is empty. Cannot remove." << std::endl;  
        return;  
    }  
    Node *temp = head;  
    head = head->next;  
    delete temp;  
    if (head == nullptr) {  
        tail = nullptr; // List became empty  
    }  
}
```





REMOVE FROM REAR





REMOVE FROM REAR

- **Exercise:**

- Try and write the `removeFromRear()` function.
- Make sure to take care of all special cases. In particular you will have to:
 - Think about the case when the list is empty.
 - If non-empty, will you need to adjust the tail?
 - If non-empty, will you need to adjust the head?
 - Any other special cases?





DELETION IN GENERAL





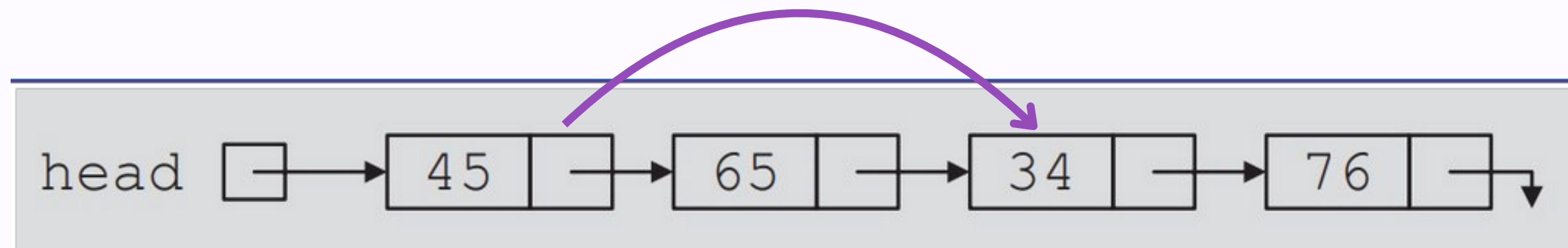
GENERAL DELETE FUNCTION

- How will you delete an arbitrary element from a linked list?
- Assume that the element already exists in the list
- Hint:
 - Find the element to be deleted.
 - Find its *previous* element.
 - `previous→next = previous→next→next;`





GENERAL DELETE FUNCTION



```
previous→next = previous→next→next;
```





THE FIND FUNCTION





THE FIND FUNCTION

- Observe that to delete a given element from the linked list (as shown in the previous slide), we need to first find it within the linked list.
- The declaration of the find function could be:
 - `Node *find(int value);`
- Try writing the code for this find function.





OTHER LINKED DATA STRUCTURES





OTHER LINKED DATA STRUCTURES

- Apart from Linked Lists, there are other linked data structures as well that can be:
 - created from the class Node,
 - or created using the LinkedList class.
- For example:
 - Queues,
 - Stacks,
 - Double ended queues (or Dequeues)





SUMMARY





SUMMARY

- This session was an introduction to the linked list data structure.
- The linked list data structure is a linked data structure based on the idea of nodes - data elements with a value and a link.
- Linked list typically support the add, delete, find and remove functions at the two ends of the list.
- Other linked data structures can also be made such as stacks, queues, and double ended queues (also called dequeues).





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 06





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

